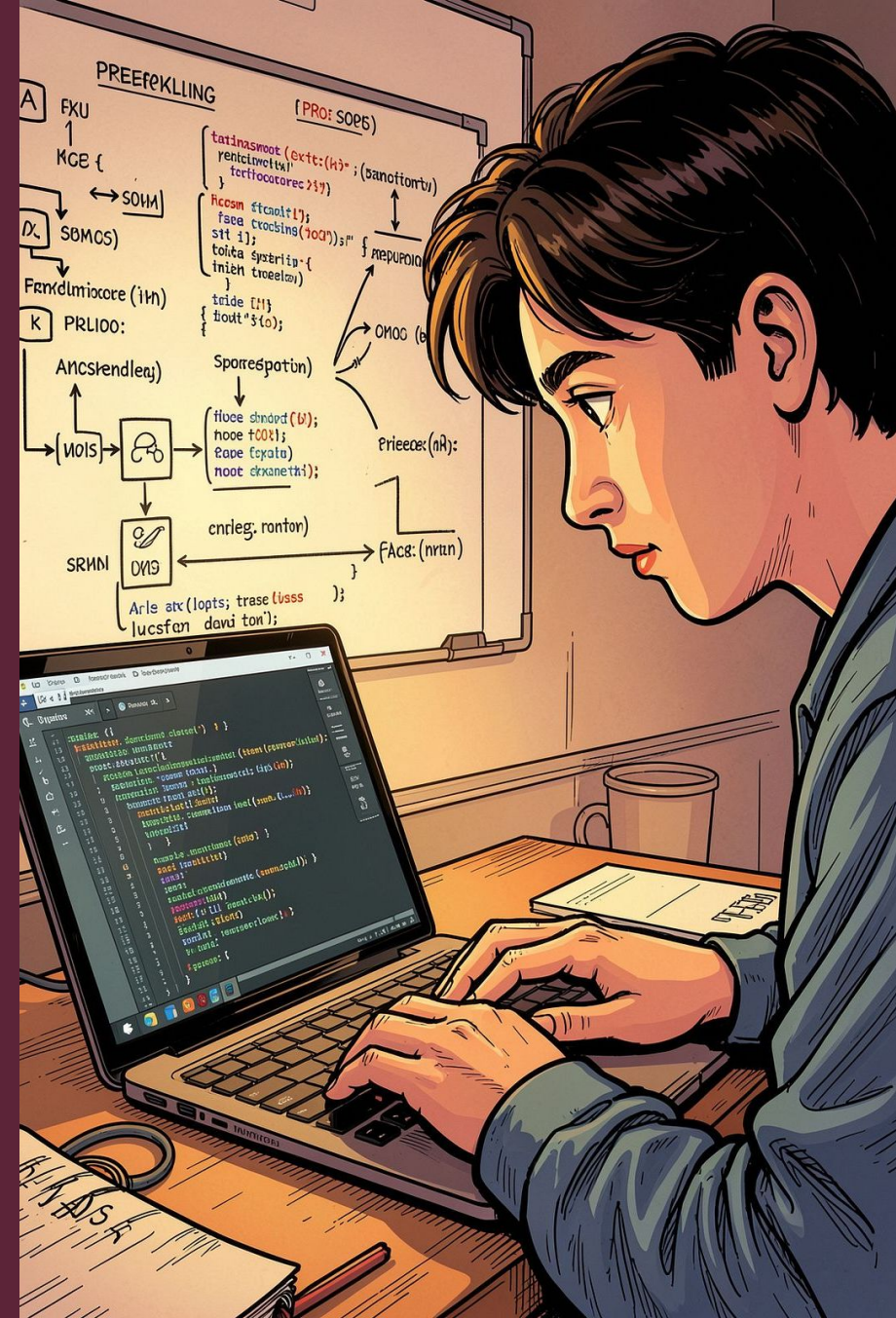


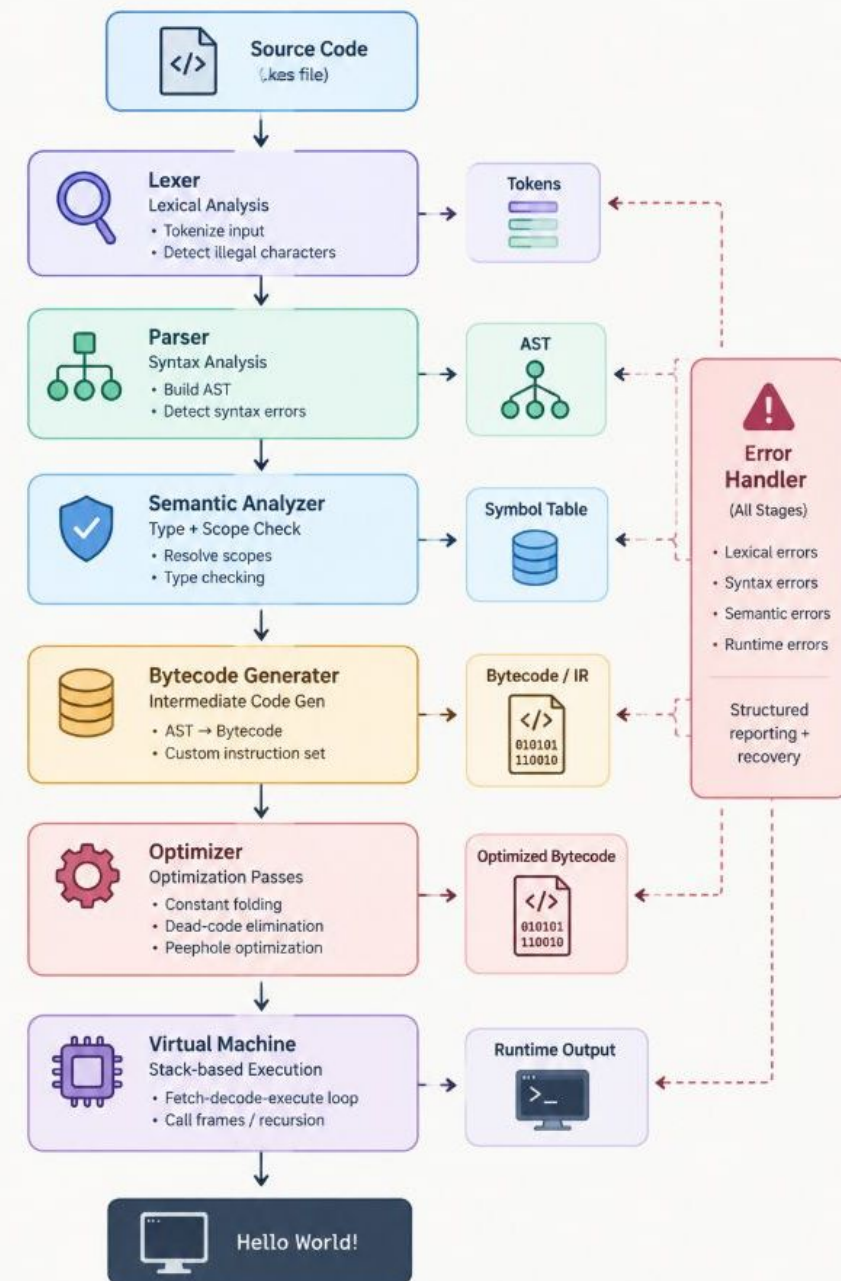
Kestrel: Design & Implementation of a Stack-Based Bytecode Compiler and VM

Phase 1 — Compiler Design Laboratory: Problem Definition & Design. A compact, end-to-end compiler pipeline intended as a student project that covers lexing, parsing, AST construction, and structured error reporting, with a roadmap toward bytecode and a stack VM.



Abstract

Kestrel is a small procedural language and a complete compiler pipeline built for hands-on education. It supports variables, arithmetic/logical expressions, conditionals, loops, and functions (planned). Phase 1 delivers a lexer and recursive-descent parser that emit a token stream and an AST, plus structured error reporting for lexical and syntax faults.



Problem Statement & Motivation

Problem

Intro compilers often show isolated stages (e.g., expression evaluators) or remain theoretical. Students need a manageable, defensible project that spans the full pipeline and illustrates interactions between stages.

Motivation

- Bridge theory and practice: front end ↔ execution model
- Expose runtime concepts (call frames, scope, stack discipline)
- Teach production patterns: recursive-descent parsing and Visitor AST traversal

Objectives

Language Spec

Formal grammar, tokens, and precedence rules

Lexer

Handwritten tokenizer with lexical errors

Parser & AST

Recursive-descent parser producing AST nodes (Visitor)

Semantic Analysis

Scope resolution, type checks, undeclared detection

Bytecode & VM

Custom IR, optimizer passes, stack VM with call frames

Error Handling

Structured errors across all stages, recovery in parser

Scope (In / Out)

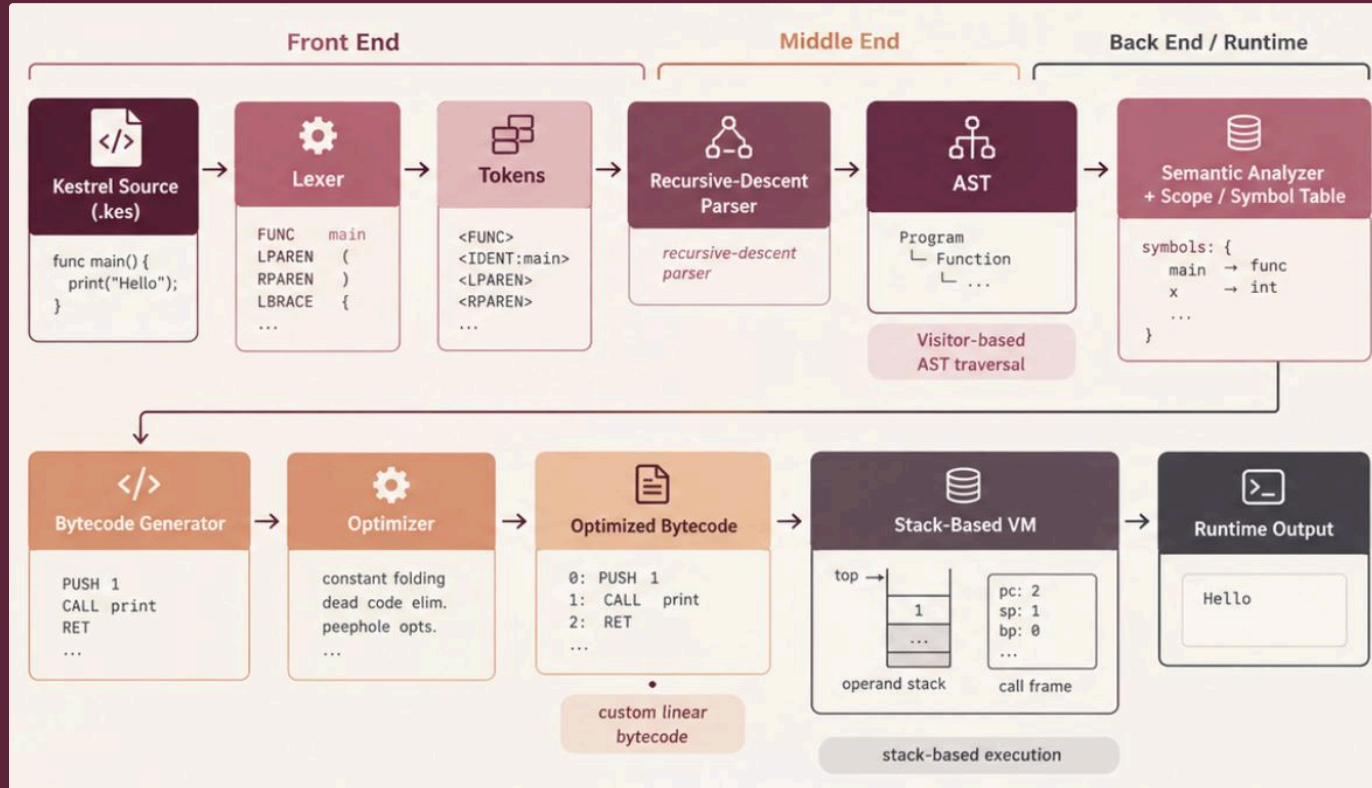
In Scope

- Procedural language: vars, expressions, if/while, functions (Phase 2)
- Full pipeline: source → bytecode → VM
- Nested-scope symbol table and three optimizations
- Structured error reporting and recovery

Out of Scope

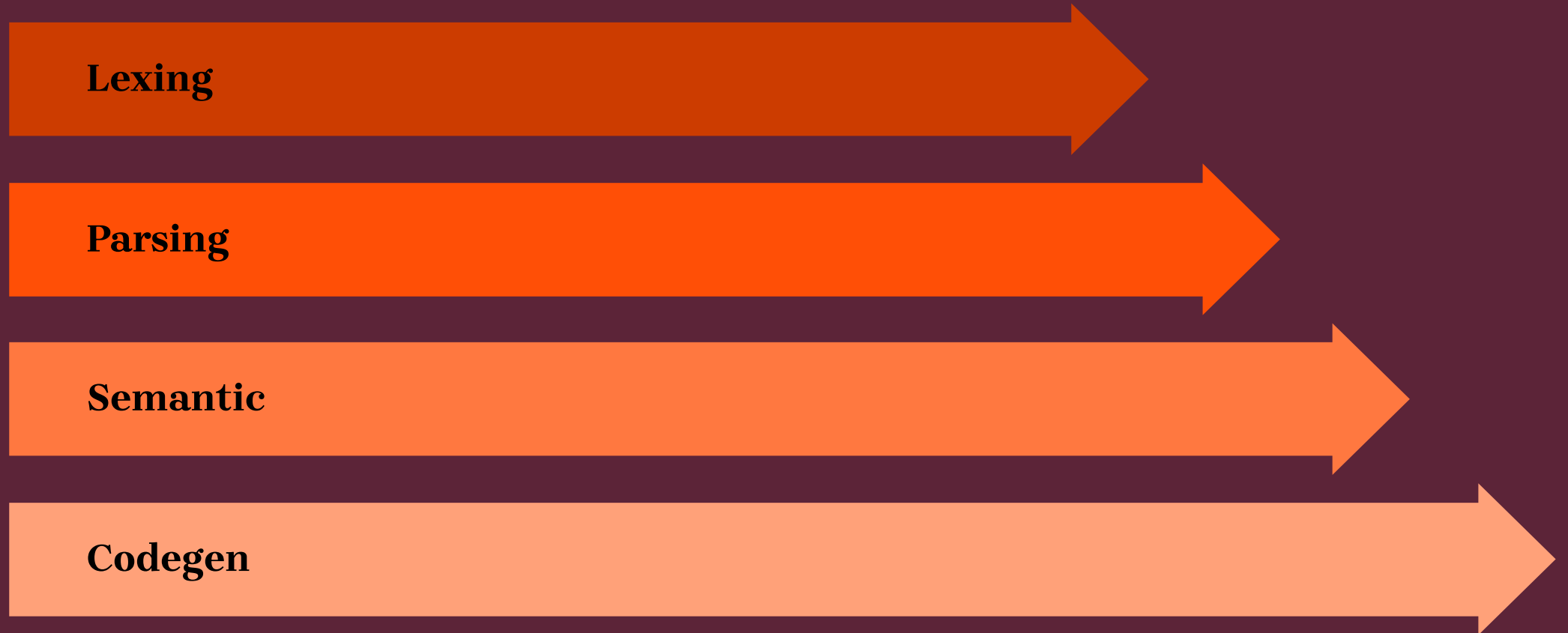
- No native code generation or JVM bytecode
- No advanced type system (generics/classes)
- No multithreading, networking, or GUI IDE

Background & Design Choices



Classical pipeline adopted: lexical → syntax → semantic → IR → optimization → target bytecode → VM execution. Kestrel uses a custom linear bytecode and a stack-based VM to clearly separate analysis from execution and to make optimization effects measurable. The parser is recursive-descent; AST traversal uses the Visitor pattern for modular passes.

System Architecture (Pipeline)

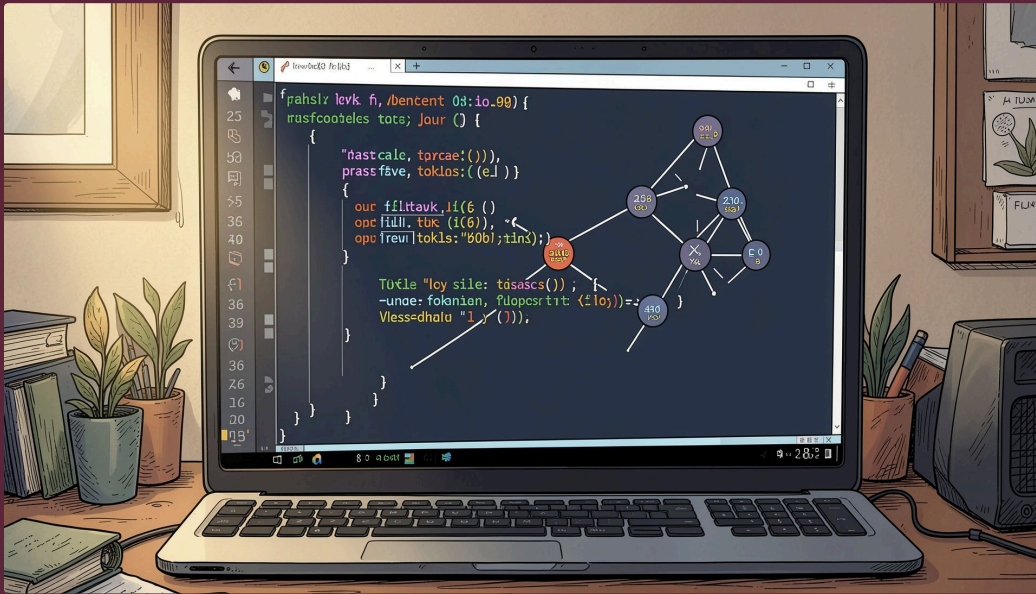


Each node represents a discrete pass. The Error Handler attaches metadata (stage, line, message) and allows the parser to recover to a synchronization point rather than abort on first error.

Compiler Concepts — Realized in Kestrel

Concept Area	How It's Realised in Kestrel
Lexical Analysis	Hand-written Lexer tokenizing identifiers, keywords, numbers, strings; detects illegal chars and unterminated strings
Syntax Analysis	Recursive-descent Parser implementing EBNF grammar, emitting an AST with line-numbered syntax errors and recovery
Semantic Analysis	AST pass for scope resolution, undeclared-variable checks, and basic static type checks
Symbol Table Management	Nested-scope table (global, function, block) supporting shadowing and type annotations
Intermediate Code Generation	AST-to-bytecode lowering producing a linear instruction stream (stack-based semantics)
Code Optimization	Bytecode-level constant folding, dead-code elimination, peephole optimization with before/after benchmarks
Target Code Generation	Final optimized bytecode emitted for the custom VM
Interpretation & Execution	Stack-based VM with fetch-decode-execute loop and call frames enabling recursion
Error Detection & Recovery	Structured CompilerError objects (stage, line, message); parser sync points avoid cascading failures

Phase 1 Prototype — Implementation Summary



Core Components

- TokenType.java & Token.java — token model
- Lexer.java — hand-written scanner + lexical errors
- Expr.java / Stmt.java — AST node hierarchy (Visitor)
- Parser.java — recursive-descent implementation with error recovery
- AstPrinter.java — readable indented parse tree output
- CompilerError.java — shared error type with stage & line
- Main.java — CLI: input .kes -> tokens + parse tree

Sample Input & Behavior

```
Source: x = 5; y = 10; z = (x + y) * 2 - 3; print(z);
```

Result: AST reflects correct precedence: subtraction node with left child multiplication (parenthesized addition inside multiplication).

Error demo: `if (x < 10 {` produces: **[line 3] Syntax Error: Expected ')' after if condition (found '{')** — parser recovers instead of crashing.

Methodology, Technology Stack & Next Steps

9-Step Methodology

1. Language specification (grammar & tokens)
2. Lexer implementation & tests
3. Parser & AST construction
4. Semantic analysis & symbol table
5. Bytecode instruction set design
6. Implement optimizations & benchmarks
7. Implement stack VM with call frames
8. Integration, system tests, performance collection
9. Documentation, lab report, presentation

📌 Phase 1 completed: steps 1–3 prototyped. Phase 2 focuses on semantic analysis, bytecode, optimizer, and VM.

Technology Stack

Implementation	Java (JDK 21)
Build	javac / JDK toolchain
VCS	Git + GitHub
Testing	Custom sample-program test suite (valid/invalid/boundary)
Dev Env	VS Code / IntelliJ IDEA
Docs	Markdown and final report (Word)

Next Steps (Phase 2)

- Extend grammar for function declarations, calls, returns
- Implement semantic pass and nested-scope symbol table
- Design IR and implement bytecode generator
- Implement optimizer passes and measure performance
- Develop stack-based VM: call frames, recursion, runtime errors